

test-monkey例程

将如下文件保存为filament/samples/test-monkey.cpp

```
#include <SDL.h>
#include <filament/Engine.h>
#include <filament/Camera.h>
#include <filament/FilamentAPI.h>
#include <filament/LightManager.h>
#include <filament/Material.h>
#include <filament/MaterialInstance.h>
#include <filament/RenderableManager.h>
#include <filament/Scene.h>
#include <filament/View.h>
#include <filament/Skybox.h>
#include <filament/Viewport.h>
#include <filament/Renderer.h>
#include <filament/VertexBuffer.h>
#include <filament/IndexBuffer.h>
#include <utils/EntityManager.h>
#include <filament/TransformManager.h>
#include <thread>
#include <utils/EntityManager.h>
#include <filament/TransformManager.h>
#include <math/norm.h>

#include <vector>
#include <fstream>
#include <iostream>
#include "filamentapp/MeshAssimp.h"

using namespace filament;
using namespace utils;
using namespace math;
using namespace std;

SDL_Window* createSDLwindow(int x, int y, int w, int h) {

    const uint32_t windowFlags =
        SDL_WINDOW_SHOWN | SDL_WINDOW_RESIZABLE | SDL_WINDOW_ALLOW_HIGHDPI;
    SDL_Window* win =
        SDL_CreateWindow("Hello World!", x, y, w, h, windowFlags);
    if (win == nullptr) {
        std::cout << "SDL_CreateWindow Error: " << SDL_GetError() << std::endl;
        SDL_Quit();
        return NULL;
    }

    return win;
}

#include <utils/Panic.h>
#include <SDL_syswm.h>

void* getNativeWindow(SDL_Window* sdlWindow) {
    SDL_SysWMInfo wmi;
    SDL_VERSION(&wmi.version);
    ASSERT_POSTCONDITION(SDL_GetWindowWMInfo(sdlWindow, &wmi), "SDL version unsupported!");
    if (wmi.subsystem == SDL_SYSWM_X11) {
#ifdef FILAMENT_SUPPORTS_X11
        Window win = (Window) wmi.info.x11.window;
        return (void *) win;
#endif
    }
    else if (wmi.subsystem == SDL_SYSWM_WAYLAND) {
#ifdef FILAMENT_SUPPORTS_WAYLAND

```

```

        int width = 0;
        int height = 0;
        SDL_GetWindowSize(sdlWindow, &width, &height);

        // Static is used here to allocate the struct pointer for the lifetime of the program.
        // Without static the valid struct quickly goes out of scope, and ends with seemingly
        // random segfaults.
        static struct {
            struct wl_display *display;
            struct wl_surface *surface;
            uint32_t width;
            uint32_t height;
        } wayland {
            wmi.info.wl.display,
            wmi.info.wl.surface,
            static_cast<uint32_t>(width),
            static_cast<uint32_t>(height)
        };
        return (void *) &wayland;
#endif
    }
    return nullptr;
}

std::ifstream::pos_type getFileSize(const char* filename) {
    std::ifstream in(filename, std::ifstream::ate | std::ifstream::binary);
    return in.tellg();
}

int main(int argc, char** argv) {
    int winPosX = 100;
    int winPosY = 100;
    int winWidth = 1280;
    int winHeight = 720;
    SDL_Window* window = createSDLWindow(winPosX, winPosY, winWidth, winHeight);
    Engine* engine = Engine::create();
    SwapChain* swapChain = engine->createSwapChain(getNativeWindow(window));
    Renderer* renderer = engine->createRenderer();

    Entity cameraEntity = utils::EntityManager::get().create();
    Camera* camera = engine->createCamera(cameraEntity);
    View* view = engine->createView();
    Scene* scene = engine->createScene();

    view->setCamera(camera);
    // Determine the current size of the window in physical pixels.
    uint32_t w, h;
    SDL_GL_GetDrawableSize(window, (int*)&w, (int*)&h);

    camera->setProjection(90.0, double(w) / h, 0.1, 50, Camera::Fov::VERTICAL);
    view->setViewport({ 0, 0, w, h });

    view->setScene(scene);

    std::string filename = "generated/material/aiDefaultMat.filamat";
    long fileSize = static_cast<long>(getFileSize(filename.c_str()));
    if (fileSize <= 0) {
        return 1;
    }

    Material* material = nullptr;
    MaterialInstance* materialInstance = nullptr;
    std::ifstream in(filename.c_str(), std::ifstream::in);
    if (in.is_open()) {

        std::vector<char> materialBuffer;
        materialBuffer.reserve(static_cast<unsigned long>(fileSize));

        in.read(materialBuffer.data(), fileSize);
        material = Material::Builder()
            .package((void*)materialBuffer.data(), (size_t)fileSize)

```

```

        .build(*engine);
materialInstance = material->createInstance();

materialBuffer.clear();

}

Entity light = EntityManager::get().create();

LightManager::Builder(LightManager::Type::SUN)
    .color(Color::toLinear<ACCURATE>(sRGBColor(0.98f, 0.92f, 0.89f)))
    .intensity(110000)
    .direction({ 0.7, -1, -0.8 })
    .sunAngularRadius(1.9f)
    .castShadows(true)
    .build(*engine, light);

scene->addEntity(light);

string modelFilename = "assets/models/monkey/monkey.obj";

std::map<std::string, MaterialInstance*> materialInstanceMaps;
materialInstanceMaps["DefaultMaterial"] = materialInstance;
materialInstanceMaps["DefaultMaterial"]->setParameter("baseColor", float3{ 0.0f, 0.8f, 0.8f });
materialInstanceMaps["DefaultMaterial"]->setParameter("metallic", 0.0f);
materialInstanceMaps["DefaultMaterial"]->setParameter("roughness", 0.4f);
materialInstanceMaps["DefaultMaterial"]->setParameter("reflectance", 0.5f);

std::unique_ptr<MeshAssimp> meshSet = std::make_unique<MeshAssimp>(*engine);
meshSet->addFromFile(modelFilename, materialInstanceMaps, true);

mat4f origTransform = math::mat4f::translation(float3{ 0, -1, -100 }) * mat4f::rotation(0.5 * M_PI,
float3{ 1, 0, 0 });
auto& rcm = engine->getRenderableManager();
auto& tcm = engine->getTransformManager();
for (auto renderable : meshSet->getRenderables()) {
    if (rcm.hasComponent(renderable)) {
        auto ti = tcm.getInstance(renderable);
        tcm.setTransform(ti, mat4f{ mat3f(1), float3(0.0f, 0.0f, -4.0f) } *
            tcm.getWorldTransform(ti));
        scene->addEntity(renderable);
    }
}

bool run = true;
double g_discoAngle = 0;
double g_discoAngularSpeed = 1;
double lastTime = 0;

float3 pos = 0;
while (run) {
    if (renderer->beginFrame(swapChain)) {
        renderer->render(view);
        renderer->endFrame();
    }
    double now = (double)SDL_GetPerformanceCounter() / SDL_GetPerformanceFrequency();
    double dT = now - lastTime;
    SDL_Event event;
    int numEvents = 0;
    while (SDL_PollEvent(&event) && numEvents < 16) {

        switch (event.type) {
            case SDL_QUIT:
                run = false;
                break;
        }
        numEvents++;
    }

    SDL_Delay(16);
    g_discoAngle += g_discoAngularSpeed * dT;
}

```

```

        lastTime = now;
    }

    // ----- clear
    for (auto& item : materialInstanceMaps) {
        auto materialInstance = item.second;
        engine->destroy(materialInstance);
    }
    meshSet.reset(nullptr);
    engine->destroy(material);

    engine->destroy(view);
    engine->destroy(scene);
    engine->destroy(renderer);
    engine->destroyCameraComponent(cameraEntity);

    EntityManager& em = EntityManager::get();
    engine->destroy(light);
    em.destroy(light);

    Engine::destroy(&engine);
    engine = nullptr;

    return 0;
}

```

在add_demo部分后面补充对应的选项:

```

add_demo(viewtest)
add_demo(test-monkey)

```

编译即可运行。

目前只在ubuntu 20.04测试通过。